

SJF: Shortest Job First

Implemented rules and examples for Scheduling Studio. These describe the current visualizer, including its deterministic tie and boundary ordering.

Model and terminology

The visualizer uses one CPU, discrete ticks, and CPU-only processes. A process has an arrival time and a service time. Service is the total CPU work required to finish; remaining service is the work still needed. Waiting does not consume service. There is no I/O, blocking, or context-switch cost in the normal UI.

A snapshot at time t shows the state after the scheduling decisions at t and before execution during $[t, t + 1)$. If a process runs during that interval, its remaining service decreases by one. Completion is recorded at the following boundary. Finished processes remain in the metrics table but leave the CPU and ready queue.

1. Admit arrivals in stable order

Arrivals join the back of the ready queue. Simultaneous arrivals retain input order. A future process is not eligible for selection.

Example: If B and C arrive at time 2 in that input order, they enter as B, C. Their service lengths decide which is selected when the CPU next becomes free.

2. Select the shortest original service when the CPU is free

Among all ready processes, choose the one with the smallest original service time. The displayed queue retains insertion order; selection can remove a process from the middle or end. It is not restricted to the displayed queue head.

Example: Ready order B, C with services six and one selects C.

3. Break dispatch ties by arrival, then input order

For equal service lengths, prefer the earlier arrival. If arrival times also match, prefer the earlier input position. IDs are not used to break ties.

Example: X runs during 0–5. C arrives at 2 and B at 1; both need two ticks. At 5, B runs first even if C was listed first in the input.

4. Do not preempt a running process

This SJF implementation is non-preemptive. Once selected, a process runs until it finishes. A shorter arrival waits. There is no time slice, allotment, demotion, or boost.

Example: A starts at zero with five service ticks. C arrives at one with one service tick. A still runs until time 5; C cannot interrupt it.

5. Complete, admit arrivals, then select

At each boundary:

1. Remove a completed runner.
2. Append this boundary's arrivals in input order.
3. If the CPU is free, select the shortest ready service using the tie rules; otherwise keep the current runner.
4. Record the state, then execute the next tick unless everything has finished.

Example: A finishes at 5. B is waiting with three service ticks and C arrives at 5 with one. C is eligible immediately and runs next.

6. Handle idle periods and long waits

Record idle ticks only when no process is running or ready. Do not reserve the CPU for a shorter process that has not arrived yet. There is no aging rule: waiting longer does not override a strictly shorter ready service. All accepted finite workloads eventually complete.

Example: A is ready at zero with five ticks and B arrives at one with one tick. Start A at zero; do not wait for B.

Worked example

The following independent example uses input order A, B, C:

Process	Arrival	Service
A	0	5
B	1	2
C	1	1

Interval	CPU	Reason
0–5	A	Only A is ready at zero; later arrivals do not preempt it.
5–6	C	C's original service of one is shorter than B's two.
6–8	B	B is the remaining ready process.

Per-tick CPU sequence: A A A A A C B B. At time 8, all processes are finished.

Process	Response	Waiting	Turnaround
A	0	0	5
B	5	5	7
C	4	4	5

Metrics and display

- **Response time:** first CPU start minus arrival time. It is recorded once and remains unchanged afterward.
- **Waiting time:** total ticks spent ready but not running. Time before arrival is excluded. At completion, waiting equals turnaround minus original service.
- **Turnaround time:** completion time minus arrival time. It remains unavailable until the process finishes.
- **Averages:** waiting includes every listed process, including future processes with zero waiting. Response averages include only processes that have started; turnaround averages include only finished processes. An unavailable value is shown as a dash. Non-integer averages are displayed to one decimal place.
- **Cards and timeline:** `N service left` is remaining CPU work. A timeline cell at `t` identifies the process running during `[t, t + 1)`, or idle. Animation and reverse playback visualize the same recorded schedule and do not change its decisions.

Input limits and recovery

Use 1–50 processes. IDs must be nonempty and unique after trimming whitespace and ignoring case. Arrival times must be nonnegative safe whole numbers; service times must be positive safe whole numbers. The complete workload, including idle gaps, must finish by tick 2,000. Finishing exactly at 2,000 is allowed; finishing at 2,001 is rejected.

The numeric editor floors fractional entries and clamps them to the field minimum. JSON imports and direct engine calls must pass integer validation. Invalid edits replace the visualization with a validation message; an invalid JSON import leaves the current scenario unchanged. Editing a valid scenario or changing the policy restarts playback at time zero.